

Notice that with this general formulation, one can learn the grounding of constants, functions, and predicates. The learning of the grounding of constants corresponds to the learning of *embeddings*. The learning of the grounding of functions corresponds to the learning of *generative* models or a *regression* task. Finally, the learning of the grounding of predicates corresponds to a *classification* task in Machine Learning.

In some cases, it is useful to impose some regularization (as done customarily in ML) on the set of parameters θ , thus encoding a preference on the hypothesis space Θ , such as a preference for smaller parameter values. In this case, learning is defined as follows:

$$\theta^* = \operatorname{argmax}_{\theta \in \Theta} \left(\operatorname{SatAgg}_{\phi \in \mathcal{K}} \mathcal{G}_\theta(\phi) - \lambda R(\theta) \right)$$

where $\lambda \in \mathbb{R}^+$ is the regularization parameter and R is a regularization function, e.g. L_1 or L_2 regularization, that is, $L_1(\theta) = \sum_{\theta \in \Theta} |\theta|$ and $L_2(\theta) = \sum_{\theta \in \Theta} \theta^2$.

LTN can generalize and extrapolate when querying formulas grounded with unseen data (for example, new individuals from a domain), using knowledge learned with previous groundings (for example, re-using a trained predicate). This is explained in Section 3.3.

3.3. Querying

Given a grounded theory $\mathcal{T} = (\mathcal{K}, \mathcal{G}_\theta)$, *query answering* allows one to check if a certain fact is true (or, more precisely, by how much it is true since in Real Logic truth-values are real numbers in the interval $[0,1]$). There are various types of queries that can be asked of a grounded theory.

A first type of query is called *truth queries*. Any formula in the language of $\mathcal{T}$ can be a truth query. The answer to a truth query ϕ_q is the truth value of ϕ_q obtained by computing its grounding, i.e. $\mathcal{G}_\theta(\phi_q)$. Notice that, if ϕ_q is a closed formula, the answer is a scalar in $[0, 1]$ denoting the truth-value of ϕ_q according to $\mathcal{G}_\theta$. if ϕ_q contains n free variables $x_1, \dots, x_n$, the answer to the query is a tensor of order n such that the component indexed by $i_1 \dots i_n$ is the truth-value of ϕ_q evaluated in $\mathcal{G}_\theta(x_1)_{i_1}, \dots, \mathcal{G}_\theta(x_n)_{i_n}$.

The second type of query is called *value queries*. Any term in the language of $\mathcal{T}$ can be a value query. The answer to a value query t_q is a tensor of real numbers obtained by computing the grounding of the term, i.e. $\mathcal{G}_\theta(t_q)$. Analogously to truth queries, the answer to a value query is a “tensor of tensors” if t_q contains variables. Using value queries, one can inspect how a constant or a term, more generally, is embedded in the manifold.

The third type of query is called *generalization truth queries*. With generalization truth queries, we are interested in knowing the truth-values of formulas when these are applied to a new (unseen) set of objects of a domain, such as a validation or a test set of examples typically used in the evaluation of machine learning systems. A generalization truth query is a pair $(\phi_q(x), \mathbf{U})$, where ϕ_q is a formula with a free variable x and $\mathbf{U} = (\mathbf{u}^{(1)}, \dots, \mathbf{u}^{(k)})$ is a set of unseen examples whose dimensions are compatible with those of the domain of x . The answer to the query $(\phi_q(x), \mathbf{U})$ is $\mathcal{G}_\theta(\phi_q(x))$ for x taking each value $\mathbf{u}^{(i)}$, $1 \leq i \leq k$, in $\mathbf{U}$. The result of this query is therefore a vector of $|\mathbf{U}|$ truth-values corresponding to the evaluation of ϕ_q on new data $\mathbf{u}^{(1)}, \dots, \mathbf{u}^{(k)}$.

The fourth and final type of query is *generalization value queries*. These are analogous to generalization truth queries with the difference that they evaluate a term $t_q(x)$, and not a formula, on new data $\mathbf{U}$. The result, therefore, is a vector of $|\mathbf{U}|$ values corresponding to the evaluation of the trained model on a regression task using test data $\mathbf{U}$.

3.4. Reasoning

3.4.1. Logical consequence in Real Logic

From a pure logic perspective, reasoning is the task of verifying if a formula is a logical consequence of a set of formulas. This can be achieved semantically using model theory ($\models$) or syntactically via a proof theory ($\vdash$). To characterize reasoning in Real Logic, we adapt the notion of logical consequence for fuzzy logic provided in [9]: A formula ϕ is a fuzzy logical consequence of a finite set of formulas Γ , in symbols $\Gamma \models \phi$ if for every fuzzy interpretation f , if all the formulas in Γ are true (i.e. evaluate to 1) in f then ϕ is true in f . In other words, every model of Γ is a model of ϕ . A direct application of this definition to Real Logic is not practical since in most practical cases the level of satisfiability of a grounded theory $(\mathcal{K}, \mathcal{G}_\theta)$ will not be equal to 1. We therefore define an interval $[q, 1]$ with $\frac{1}{2} < q < 1$ and assume that a formula is true if its truth-value is in the interval $[q, 1]$. This leads to the following definition:

Definition 6. A closed formula ϕ is a logical consequence of a knowledge-base $(\mathcal{K}, \mathcal{G}(\cdot \mid \theta), \Theta)$, in symbols $(\mathcal{K}, \mathcal{G}(\cdot \mid \theta), \Theta) \models_q \phi$, if, for every grounded theory $\langle \mathcal{K}, \mathcal{G}_\theta \rangle$, if $\text{SatAgg}(\mathcal{K}, \mathcal{G}_\theta) \geq q$ then $\mathcal{G}_\theta(\phi) \geq q$.

3.4.2. Reasoning by optimization

Logical consequence by direct application of Definition 6 requires querying the truth value of ϕ for a potentially infinite set of groundings. Therefore, we consider in practice the following directions:

Reasoning Option 1 (Querying after learning). This is approximate logical inference by considering only the grounded theories that maximally satisfy $(\mathcal{K}, \mathcal{G}(\cdot \mid \theta), \Theta)$. We therefore define that ϕ is a *brave logical consequence* of a Real Logic knowledge-base $(\mathcal{K}, \mathcal{G}(\cdot \mid \theta), \Theta)$ if $\mathcal{G}_{\theta^*}(\phi) \geq q$ for all the θ^* such that:

$$\theta^* = \underset{\theta}{\operatorname{argmax}} \text{SatAgg}(\mathcal{K}, \mathcal{G}_\theta) \quad \text{and} \quad \text{SatAgg}(\mathcal{K}, \mathcal{G}_{\theta^*}) \geq q$$

The objective is to find all θ^* that optimally satisfy the knowledge base and to measure if they also satisfy ϕ . One can search for such θ^* by running multiple optimizations with the objective function of Section 3.2.

This approach is somewhat naive. Even if we run the optimization multiple times with multiple parameter initializations (to, hopefully, reach different optima in the search space), the obtained groundings may not be representative of other optimal or close-to-optimal groundings. In Section 4.8, we give an example that shows the limitations of this approach and motivates the next one.

Reasoning Option 2 (Proof by Refutation). Here, we reason by refutation and search for a counter-example to the logical consequence by introducing an alternative search objective. Normally, according to Definition 6, one tries to verify that:¹¹

$$\text{for all } \theta \in \Theta, \text{ if } \mathcal{G}_\theta(\mathcal{K}) \geq q \text{ then } \mathcal{G}_\theta(\phi) \geq q. \quad (21)$$

Instead, we solve the dual problem:

$$\text{there exists } \theta \in \Theta \text{ such that } \mathcal{G}_\theta(\mathcal{K}) \geq q \text{ and } \mathcal{G}_\theta(\phi) < q. \quad (22)$$

¹¹For simplicity, we temporarily define the notation $\mathcal{G}(\mathcal{K}) := \text{SatAgg}_{\phi \in \mathcal{K}}(\mathcal{K}, \mathcal{G})$.

If Eq.(22) is true then a counterexample to Eq.(21) has been found and the logical consequence does not hold. If Eq.(22) is false then no counterexample to Eq.(21) has been found and the logical consequence is assumed to hold true. A search for such parameters θ (the counterexample) can be performed by minimizing $\mathcal{G}_\theta(\phi)$ while imposing a constraint that seeks to invalidate results where $\mathcal{G}_\theta(\mathcal{K}) < q$. We therefore define:

$$\text{penalty}(\mathcal{G}_\theta, q) = \begin{cases} c \text{ if } \mathcal{G}_\theta(\mathcal{K}) < q, \\ 0 \text{ otherwise,} \end{cases} \quad \text{where } c > 1.^{12}$$

Given $\mathcal{G}^*$ such that:

$$\mathcal{G}^* = \underset{\mathcal{G}_\theta}{\operatorname{argmin}}(\mathcal{G}_\theta(\phi) + \text{penalty}(\mathcal{G}_\theta, q)) \quad (23)$$

- If $\mathcal{G}^*(\mathcal{K}) < q$: Then for all $\mathcal{G}_\theta$, $\mathcal{G}_\theta(\mathcal{K}) < q$ and therefore $(\mathcal{K}, \mathcal{G}(\cdot | \theta), \Theta) \models_q \phi$.
- If $\mathcal{G}^*(\mathcal{K}) \geq q$ and $\mathcal{G}^*(\phi) \geq q$: Then for all $\mathcal{G}_\theta$ with $\mathcal{G}_\theta(\mathcal{K}) \geq q$, we have that $\mathcal{G}_\theta(\phi) \geq \mathcal{G}^*(\phi) \geq q$ and therefore $(\mathcal{K}, \mathcal{G}(\cdot | \theta), \Theta) \models_q \phi$.
- If $\mathcal{G}^*(\mathcal{K}) \geq q$ and $\mathcal{G}^*(\phi) < q$: Then $(\mathcal{K}, \mathcal{G}(\cdot | \theta), \Theta) \not\models_q \phi$.

Clearly, Equation (23) cannot be used as an objective function for gradient-descent due to null derivatives. Therefore, we propose to approximate the penalty function with the soft constraint:

$$\text{elu}(\alpha, \beta(q - \mathcal{G}_\theta(\mathcal{K}))) = \begin{cases} \beta(q - \mathcal{G}_\theta(\mathcal{K})) & \text{if } \mathcal{G}_\theta(\mathcal{K}) \leq q, \\ \alpha(e^{q - \mathcal{G}_\theta(\mathcal{K})} - 1) & \text{otherwise,} \end{cases}$$

where $\alpha \geq 0$ and $\beta \geq 0$ are hyper-parameters (see Figure 6). When $\mathcal{G}_\theta(\mathcal{K}) < q$, the penalty is linear in $q - \mathcal{G}_\theta(\mathcal{K})$ with a slope of β . Setting β high, the gradients for $\mathcal{G}_\theta(\mathcal{K})$ will be high in absolute value if the knowledge-base is not satisfied. When $\mathcal{G}_\theta(\mathcal{K}) > q$, the penalty is a negative exponential that converges to $-\alpha$. Setting α low but non-zero seeks to ensure that the gradients do not vanish when the penalty should not apply (when the knowledge-base is satisfied). We obtain the following approximate objective function:

$$\mathcal{G}^* = \underset{\mathcal{G}_\theta}{\operatorname{argmin}}(\mathcal{G}_\theta(\phi) + \text{elu}(\alpha, \beta(q - \mathcal{G}_\theta(\mathcal{K}))) \quad (24)$$

Section 4.8 will illustrate the use of reasoning by refutation with an example in comparison with reasoning as *querying after learning*. Of course, other forms of reasoning are possible, not least that adopted in [6], but a direct comparison is outside the scope of this paper and left as future work.

4. The Reach of Logic Tensor Networks

The objective of this section is to show how the language of Real Logic can be used to specify a number of tasks that involve learning from data and reasoning. Examples of such tasks are classification, regression, clustering, and link prediction. The solution of a problem specified in Real Logic is obtained by interpreting such a specification in *Logic Tensor Networks*. The LTN library implements Real Logic in Tensorflow 2 [1] and is available from GitHub¹³. Every logical operator

¹²In the objective function, $\mathcal{G}^*$ should satisfy $\mathcal{G}^*(\mathcal{K}) \geq q$ before reducing $\mathcal{G}^*(\phi)$ because the penalty c which is greater than 1 is higher than any potential reduction in $\mathcal{G}(\phi)$ which is smaller or equal to 1.

¹³<https://github.com/logictensornetworks/logictensornetworks>